TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS · CAPÍTULO 4

DDL, restrições e JOIN

Criar e alterar tabelas em SQL, proteger as ligações e juntar os dados

```sql
SELECT a.nome_aluno, d.nome_disc, m.nota
FROM matriculas m
JOIN alunos      a ON a.rm_aluno   = m.rm_aluno
JOIN disciplinas d ON d.sigla_disc = m.sigla_disc;
```

PROFESSOR
Tarik Ponciano

Roteiro

01 **DDL: a estrutura em SQL**
CREATE, ALTER e DROP — escrever o que o pgAdmin gerava nos cliques

02 **Restrições: FOREIGN KEY e CHECK**
O banco passa a cobrar as ligações e as regras do negócio

03 **Do MER ao script**
Implementar um DER/MER inteiro num arquivo `.sql`

04 **JOIN**
Juntar tabelas para responder perguntas de verdade

★ **Consolidação**
Do briefing ao JOIN: a Academia Pulso

NESTE MATERIAL

Exemplo explicado
Um caso resolvido passo a passo.

Prática
Individual e digital, no seu PostgreSQL, com o professor acompanhando.

Código SQL
Testado no PostgreSQL, com o resultado e as mensagens que aparecem na tela.

ONDE ESTAMOS

O que ficou pendente no Capítulo 3

NO CAPÍTULO 3

As tabelas foram feitas no clique

O pgAdmin escreveu o `CREATE TABLE` por nós, na aba SQL.

HOJE • SEÇÃO 01

DDL: escrever, alterar e apagar a estrutura em SQL.

NO CAPÍTULO 3

A matrícula do aluno 999 passou

E excluir o André deixou as matrículas dele órfãs. Nota 15? Também passaria.

HOJE • SEÇÕES 02 E 03

FOREIGN KEY e **CHECK** — e o MER inteiro num script.

NO CAPÍTULO 3

A matrícula só mostra códigos

```
cod_mat | rm_aluno | sigla_disc | nota
      1 |      101 | BD         | 8.50
```

HOJE • SEÇÃO 04

JOIN: trazer o nome do aluno e da disciplina junto.

No fim do capítulo, você sai do briefing de um cliente e chega a um banco com regras, dados e consultas que cruzam várias tabelas.

SEÇÃO 01

DDL: a estrutura em SQL

01

CREATE, ALTER e DROP — escrever o que o pgAdmin gerava nos cliques.

01 • DDL

Clicar ou escrever?

No Capítulo 3, cada clique no pgAdmin virava um comando na aba **SQL**. Aquilo era **DDL** — a família que define a estrutura do banco.

```
CREATE TABLE IF NOT EXISTS public.alunos
(
  rm_aluno integer NOT NULL,
  nome_aluno character varying(100) NOT NULL,
  PRIMARY KEY (rm_aluno)
);
```

`CREATE` cria `ALTER` altera `DROP` apaga

POR QUE ESCREVER O DDL

Repetível

O mesmo arquivo monta o banco no seu PC, no laboratório e na Aiven.

Guardável

Um arquivo `.sql` vai para o GitHub junto com o projeto.

Revisável

Um colega lê o script e acha o erro antes de rodar.

Rápido

Uma tabela inteira em um comando — sem abrir aba por aba.

No mercado, a estrutura do banco quase sempre vive em scripts. O pgAdmin continua ótimo para **conferir** o que o script criou.

CREATE TABLE: o molde

```
CREATE TABLE nome_tabela (
  coluna1  tipo  [restrições],
  coluna2  tipo  [restrições],
  …
  colunaN  tipo  [restrições]
  [, restrições da tabela]
);
```

MAIÚSCULA escreva igual *nome* troque pelo seu […] opcional

- Uma coluna por linha: **nome**, **tipo** e, se houver, as **restrições**.
- Colunas separadas por **vírgula** — a última não leva.
- Tudo entre **parênteses**, e `;` no fim.
- Restrições: `PRIMARY KEY` `NOT NULL` `UNIQUE` `DEFAULT` — as mesmas do Capítulo 3, agora escritas.
- Nomes em minúsculas, com `_`, sem acento: a regra da turma continua.

```
CREATE TABLE   ← a tabela foi criada
```

EXEMPLO EXPLICADO • CREATE TABLE

Do clique ao comando: a tabela professores

```
CREATE TABLE professores (
  cod_prof   integer      PRIMARY KEY,
  nome_prof  varchar(100) NOT NULL,
  email_prof varchar(100) UNIQUE
);
```

Alinhar as colunas com espaços não muda nada para o banco — mas facilita muito a leitura.

NO PGADMIN VOCÊ…	NO SQL VOCÊ ESCREVE
Name: **professores**	`CREATE TABLE professores`
Data type **character varying**, Length **100**	`varchar(100)`
Ligava **Primary key?**	`PRIMARY KEY`
Ligava **Not NULL?**	`NOT NULL`
Constraints → **Unique**	`UNIQUE`
Default value **0**	`DEFAULT 0`

Depois de rodar, clique com o botão direito em **Tables** → **Refresh**: a tabela aparece na árvore, igualzinha à que você criava no clique.

Restrição na coluna ou no fim da tabela

NA COLUNA • O MAIS COMUM

```
CREATE TABLE alunos (
  rm_aluno   integer PRIMARY KEY,
  nome_aluno varchar(100) NOT NULL,
  cpf_aluno  char(11) UNIQUE
);
```

A restrição vale para **aquela** coluna. Simples e direto.

NO FIM • QUANDO ENVOLVE MAIS DE UMA COLUNA

```
CREATE TABLE matriculas (
  cod_mat    integer PRIMARY KEY,
  rm_aluno   integer NOT NULL,
  sigla_disc varchar(10) NOT NULL,
  UNIQUE (rm_aluno, sigla_disc)
);
```

O **par** não se repete: o mesmo aluno não entra duas vezes na mesma disciplina.

DAR NOME À RESTRIÇÃO • OPCIONAL

Com `CONSTRAINT nome`, a mensagem de erro fica legível — e fica fácil apagar a regra depois.

```
CONSTRAINT uq_aluno_disc
  UNIQUE (rm_aluno, sigla_disc)
```

Lembra do aviso do Capítulo 3? Com PK própria, o banco não impedia o mesmo aluno duas vezes na mesma disciplina. O `UNIQUE` do par resolve.

Código automático: serial

Com `serial`, o banco numera a PK sozinho: 1, 2, 3… Você não informa o código no INSERT.

```
CREATE TABLE avisos (
  cod_aviso serial       PRIMARY KEY,
  texto     varchar(200) NOT NULL
);

INSERT INTO avisos (texto)
VALUES ('Prova de BD na sexta'),
       ('Laboratório fechado na segunda');

INSERT INTO avisos (texto)
VALUES ('Entrega do projeto') RETURNING *;
```

RESULTADO DO RETURNING

cod_aviso	texto
3	Entrega do projeto

Não misture

Inseriu o código 4 “na mão”? O `serial` não fica sabendo — e o próximo INSERT automático tenta usar o 4:

```
ERROR: duplicate key value violates unique
constraint "avisos_pkey"
DETAIL: Key (cod_aviso)=(4) already exists.
```

Nos exercícios de hoje os códigos vêm prontos nos dados, então seguimos com `integer`. Em sistemas reais, o código automático é o padrão.

01 • DDL • ATENÇÃO

CREATE DATABASE e a Query Tool

```
CREATE DATABASE clinica;
```

1. Rode o comando numa Query Tool aberta em qualquer banco (ex.: `postgres`).
2. No Object Explorer: **Databases** → **Refresh**. O banco novo aparece.
3. Clique no banco novo e abra **uma nova Query Tool nele**. É ali que vão os `CREATE TABLE`.

O erro mais comum da aula

Criou as tabelas com a Query Tool ainda aberta no banco `postgres`? Elas foram parar lá. No banco novo:

```
ERROR: relation "pacientes" does not exist
```

Confira sempre o nome do banco no topo da Query Tool: `clinica/postgres@…`

E para apagar um banco?

`DROP DATABASE clinica;` — só funciona se ninguém estiver conectado a ele, nem a sua própria Query Tool.

DROP: apagar a estrutura

```
DROP TABLE [IF EXISTS] nome_tabela;
```

COM E SEM IF EXISTS

```
DROP TABLE eventos;
```

```
DROP TABLE
```

```
DROP TABLE eventos;    -- de novo
```

```
ERROR: table "eventos" does not exist
```

```
DROP TABLE IF EXISTS eventos;
```

```
NOTICE: table "eventos" does not exist, skipping
```

QUERO…	USO	FAMÍLIA
Apagar algumas linhas	`DELETE … WHERE`	DML
Apagar todas as linhas, manter a tabela	`DELETE FROM tabela`	DML
Apagar uma coluna	`ALTER TABLE … DROP COLUMN`	DDL
Apagar a tabela inteira, com estrutura	`DROP TABLE`	DDL
Apagar o banco inteiro	`DROP DATABASE`	DDL



Não há confirmação nem Ctrl+Z. O `DROP TABLE` leva a tabela e **todos** os dados dela.

ALTER TABLE: mudar uma tabela que já existe

```
ALTER TABLE nome_tabela  ação;
```

AÇÃO	SINTAXE	EXEMPLO
Nova coluna	`ADD COLUMN coluna tipo [restrições]`	`ADD COLUMN email_aluno varchar(100)`
Apagar coluna	`DROP COLUMN coluna`	`DROP COLUMN email_aluno`
Renomear coluna	`RENAME COLUMN antigo TO novo`	`RENAME COLUMN fone TO telefone`
Renomear tabela	`RENAME TO novo_nome`	`RENAME TO eventos_escola`
Mudar o tipo	`ALTER COLUMN coluna TYPE tipo`	`ALTER COLUMN nome_disc TYPE varchar(100)`
Valor padrão	`ALTER COLUMN coluna SET DEFAULT valor` • `DROP DEFAULT`	`ALTER COLUMN faltas SET DEFAULT 0`
Obrigatório	`ALTER COLUMN coluna SET NOT NULL` • `DROP NOT NULL`	`ALTER COLUMN cidade_aluno SET NOT NULL`
Nova regra	`ADD CONSTRAINT nome regra`	`ADD CONSTRAINT ck_nota CHECK (…)`
Apagar regra	`DROP CONSTRAINT nome`	`DROP CONSTRAINT ck_nota`

O ALTER muda a estrutura **sem perder os dados** — por isso é o comando do dia a dia de um sistema que já está em uso.

EXEMPLO EXPLICADO • ALTER TABLE

A coordenação pediu quatro mudanças

1 “Queremos o e-mail dos alunos — e cada e-mail é de um aluno só.”

```
ALTER TABLE alunos
  ADD COLUMN email_aluno varchar(100) UNIQUE;
```

Os alunos que já existem ficam com o e-mail vazio (NULL).

2 “O nome de algumas disciplinas não cabe em 60 letras.”

```
ALTER TABLE disciplinas
  ALTER COLUMN nome_disc TYPE varchar(100);
```

Aumentar o tamanho é seguro: nenhum dado deixa de caber.

3 “Disciplina nova começa com 40 horas, se ninguém informar.”

```
ALTER TABLE disciplinas
  ALTER COLUMN carga_horaria SET DEFAULT 40;
```

Vale para os próximos INSERTs; as linhas antigas não mudam.

4 “A cidade do aluno passa a ser obrigatória.”

```
ALTER TABLE alunos
  ALTER COLUMN cidade_aluno SET NOT NULL;
```

Só funciona porque todos os alunos já têm cidade preenchida.

```
ALTER TABLE   ← resposta de cada um dos quatro comandos
```

01 • DDL • ATENÇÃO

ALTER e os dados que já estão lá

A tabela já tem linhas. Toda mudança precisa fazer sentido para elas — senão o banco recusa.

TENTATIVA	MENSAGEM	COMO RESOLVER
`ADD COLUMN fone_aluno varchar(20) NOT NULL`	`column "fone_aluno" of relation "alunos" contains null values`	Coluna nova nasce vazia. Dê um `DEFAULT` junto, ou crie sem `NOT NULL`, preencha com UPDATE e só depois `SET NOT NULL`.
`ALTER COLUMN email_aluno SET NOT NULL`	`column "email_aluno" of relation "alunos" contains null values`	Preencha os vazios antes (UPDATE).
`ALTER COLUMN nome_aluno TYPE varchar(5)`	`value too long for type character varying(5)`	Diminuir só se todos os valores couberem.
`ALTER COLUMN cpf_aluno TYPE integer`	`column "cpf_aluno" cannot be cast automatically to type integer`	Trocar texto por número quase nunca é boa ideia: CPF é texto (zero à esquerda!).

Estratégia: pense no tipo e nas regras **antes** de lançar dados. Mudar a estrutura de uma tabela vazia é fácil; de uma tabela cheia, nem sempre.

PRÁTICA 1 • 10 MIN • INDIVIDUAL

Escreva o CREATE TABLE

A escola quer cadastrar os **eventos** do semestre. No banco `escola`, escreva e rode o `CREATE TABLE` — sem usar a tela de criação do pgAdmin.

COLUNA	REGRA
`cod_evento`	código; identifica o evento
`titulo`	texto até 80; obrigatório
`data_evento`	data; obrigatória
`local_evento`	texto até 60
`vagas`	número inteiro; obrigatório; se não informar, 30
`inscricoes_abertas`	verdadeiro ou falso; obrigatório; começa verdadeiro

DEPOIS DE CRIAR

1. Refresh em **Tables** e confira a aba SQL: bate com o que você escreveu?
2. Insira o evento 1: *Feira de Tecnologia*, 20/11/2026, no Pátio.
3. Insira o evento 2: *Palestra: carreira em dados*, 25/11/2026 — sem local, sem vagas, sem informar inscrições.
4. `SELECT * FROM eventos;`

PENSE E RESPONDA No evento 2, de onde vieram as vagas e o valor de `inscricoes_abertas`?

PRÁTICA 2 • 10 MIN • INDIVIDUAL

Os eventos mudaram de ideia

Na tabela `eventos` da Prática 1, com os dois eventos já cadastrados. Um comando para cada pedido.

1. Guardar o nome do **responsável** pelo evento (texto até 100).
2. Alguns títulos são longos: o **título** passa a aceitar 150 caracteres.
3. A coluna `local_evento` passa a se chamar `sala`.
4. Eventos novos começam com **40** vagas.
5. Guardar o **horário**, obrigatório, tipo `time`. Tente primeiro sem DEFAULT. O que aconteceu? Resolva com padrão `'19:00'`.
6. Desistiram do responsável: apague essa coluna.
7. A tabela passa a se chamar `eventos_escola`. Insira o evento 3, *Hackathon*, 05/12/2026, e confira com SELECT.
8. Fim do semestre: apague a tabela inteira. Rode de novo — e depois com `IF EXISTS`.

PENSE E RESPONDA No passo 7, quantas vagas tem o Hackathon? E a Feira de Tecnologia? Por que são diferentes?

SEÇÃO 02

Restrições: FOREIGN KEY e CHECK

02

O banco passa a cobrar as ligações entre as tabelas e as regras do negócio.

O banco da escola ainda aceita bobagem

No Capítulo 3, estas três operações passaram sem nenhum aviso.

ACEITO — E NÃO DEVERIA

```
INSERT INTO matriculas
  (cod_mat, rm_aluno, sigla_disc)
VALUES (7, 999, 'BD');
```

Matrícula de um aluno que **não existe**.

ACEITO — E NÃO DEVERIA

```
DELETE FROM alunos
WHERE rm_aluno = 101;
```

O André sumiu — e as matrículas dele ficaram **órfãs**.

ACEITO — E NÃO DEVERIA

```
UPDATE matriculas
SET nota = 15
WHERE cod_mat = 1;
```

Nota **15** numa prova que vale 10.

FK **FOREIGN KEY** resolve os dois primeiros: a coluna só aceita códigos que existem na tabela de origem, e o pai não sai enquanto tiver filhos.

CK **CHECK** resolve o terceiro: uma condição que todo valor precisa cumprir.

FOREIGN KEY: o molde

NA COLUNA • AO CRIAR A TABELA

```
coluna tipo REFERENCES tabela_pai (coluna_pk)
```

NO FIM DA TABELA

```
FOREIGN KEY (coluna) REFERENCES tabela_pai (coluna_pk)
```

NUMA TABELA QUE JÁ EXISTE

```
ALTER TABLE tabela_filha
  ADD CONSTRAINT nome_da_regra
  FOREIGN KEY (coluna) REFERENCES tabela_pai (coluna_pk);
```

AS REGRAS

- A FK fica no lado **N** — a mesma regra do MER.
- Mesmo **tipo** da PK de origem: `integer` com `integer`.
- A tabela pai precisa **existir antes** da filha.
- A FK **pode repetir**: vários alunos na mesma cidade, várias matrículas do mesmo aluno.
- A FK **aceita NULL**, a menos que você ponha `NOT NULL`.

EXEMPLO EXPLICADO • FOREIGN KEY

Ligando matrículas e alunos no banco da escola

1 Criamos a regra na tabela que já existe:

```
ALTER TABLE matriculas
  ADD CONSTRAINT fk_mat_aluno
  FOREIGN KEY (rm_aluno) REFERENCES alunos (rm_aluno);
```

```
ERROR: insert or update on table "matriculas" violates foreign
key constraint "fk_mat_aluno"
DETAIL: Key (rm_aluno)=(999) is not present in table "alunos".
```

O banco **não cria** a regra enquanto houver um órfão — e a mensagem diz qual é.

2 Resolvemos o órfão que sobrou dos testes do Capítulo 3:

```
DELETE FROM matriculas WHERE rm_aluno = 999;
```

3 Rodamos de novo — e ligamos as outras duas:

```
ALTER TABLE matriculas ADD CONSTRAINT fk_mat_aluno
  FOREIGN KEY (rm_aluno) REFERENCES alunos (rm_aluno);
ALTER TABLE matriculas ADD CONSTRAINT fk_mat_disc
  FOREIGN KEY (sigla_disc) REFERENCES disciplinas (sigla_disc);
ALTER TABLE disciplinas ADD CONSTRAINT fk_disc_prof
  FOREIGN KEY (cod_prof) REFERENCES professores (cod_prof);
```

```
ALTER TABLE
```

O seu banco pode ter outros órfãos — o André excluído, por exemplo. Leia o `DETAIL`, corrija aquele dado e rode o ALTER de novo.

FK em ação: o que o banco passa a recusar

TENTATIVA	MENSAGEM
Matrícula do aluno **999**, que não existe `INSERT INTO matriculas … VALUES (8, 999, 'BD')`	`violates foreign key constraint "fk_mat_aluno"` `DETAIL: Key (rm_aluno)=(999) is not present in table "alunos".`
Matrícula numa disciplina **XYZ**, que não existe	`violates foreign key constraint "fk_mat_disc"` `DETAIL: Key (sigla_disc)=(XYZ) is not present in table "disciplinas".`
Excluir o aluno **101**, que tem matrículas `DELETE FROM alunos WHERE rm_aluno = 101`	`update or delete on table "alunos" violates foreign key constraint "fk_mat_aluno" on table "matriculas"` `DETAIL: Key (rm_aluno)=(101) is still referenced from table "matriculas".`
Apagar a tabela **alunos** inteira `DROP TABLE alunos`	`cannot drop table alunos because other objects depend on it` `HINT: Use DROP ... CASCADE to drop the dependent objects too.`



“Filhos antes do pai”, do Capítulo 3, agora é **regra do banco**: para excluir o André, primeiro as matrículas dele, depois o aluno.

E os filhos, quando o pai sai?

Ao criar a FK, dá para escolher o que acontece com os filhos quando o pai é excluído.

OPÇÃO	O QUE FAZ	QUANDO USAR
sem nada • padrão	Recusa a exclusão do pai.	Quase sempre. É o mais seguro.
`ON DELETE CASCADE`	Apaga os filhos junto.	Quando o filho não faz sentido sozinho: itens de uma venda.
`ON DELETE SET NULL`	Esvazia a FK dos filhos.	Quando o filho continua existindo: disciplina sem professor.

```sql
CREATE TABLE itens_venda (
  cod_item  integer PRIMARY KEY,
  num_venda integer NOT NULL
    REFERENCES vendas (num_venda) ON DELETE CASCADE,
  qtd       integer NOT NULL
);

DELETE FROM vendas WHERE num_venda = 1;
```

```
DELETE 1   ← a venda 1 e os 2 itens dela saíram juntos
```

⚠ O `CASCADE` apaga em silêncio. Use só quando tiver certeza — nos exercícios da turma, fique com o padrão.

EXEMPLO EXPLICADO • FOREIGN KEY • ESTRATÉGIA

Configurando cada FK: cinco perguntas

1 Onde fica?
No lado **N** — na tabela filha.

2 Aponta para quem?
Para a **PK do pai**, com o **mesmo tipo**.

3 Pode ficar vazia?
Obrigatória → `NOT NULL`. Opcional → sem NOT NULL.

4 E se o pai sair?
Padrão: o banco **recusa**. CASCADE ou SET NULL só com motivo.

5 Que nome?
`fk_filha_pai` — é ele que aparece no erro.

AS TRÊS FKS DA ESCOLA

FK	APONTA PARA	VAZIA?	SE O PAI SAIR	NOME
`matriculas.rm_aluno`	alunos • `integer`	não → NOT NULL	recusa	`fk_mat_aluno`
`matriculas.sigla_disc`	disciplinas • `varchar(10)`	não → NOT NULL	recusa	`fk_mat_disc`
`disciplinas.cod_prof`	professores • `integer`	sim: disciplina ainda sem professor	recusa — ou `SET NULL`	`fk_disc_prof`

AS CINCO RESPOSTAS NO CREATE TABLE

```
cod_prof integer
  CONSTRAINT fk_disc_prof
  REFERENCES professores (cod_prof)
  ON DELETE SET NULL
```

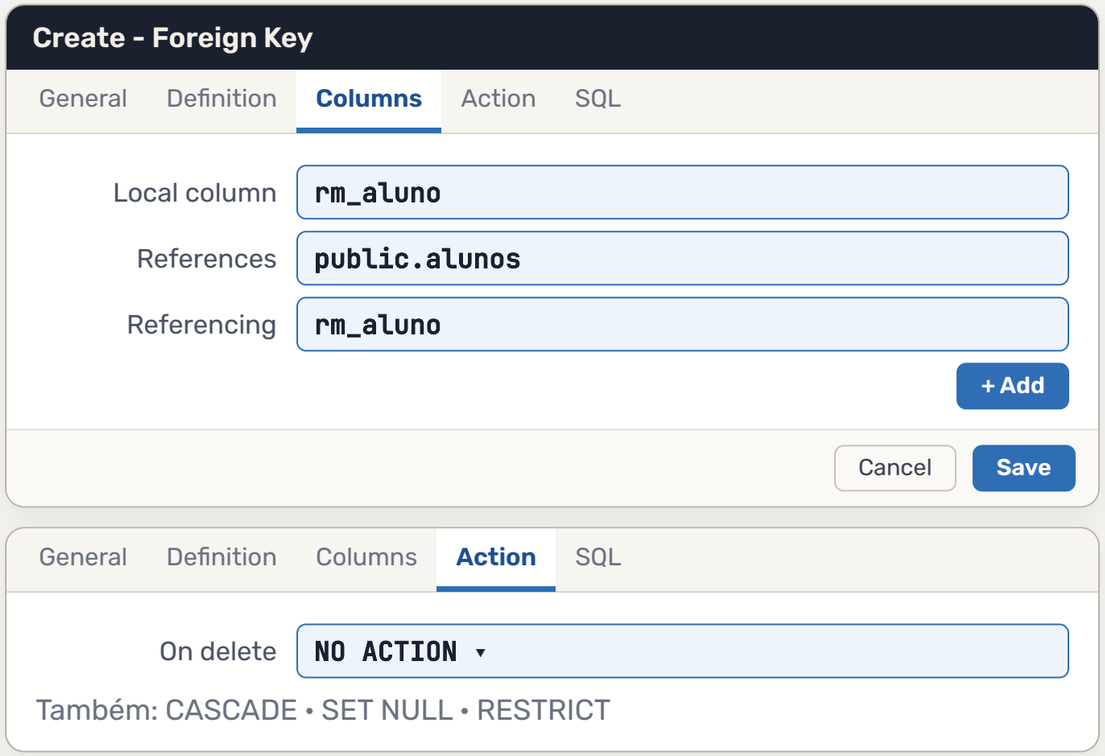
TIPO DIFERENTE NÃO LIGA

```
sigla integer REFERENCES
  disciplinas (sigla_disc)
```

```
ERROR: foreign key constraint … cannot
be implemented
DETAIL: Key columns … are of
incompatible types: integer and
character varying.
```

Sem `CONSTRAINT nome`, o PostgreSQL batiza a FK sozinho: `matriculas_rm_aluno_fkey`. Funciona — mas o nome escolhido por você é o que se lê no erro.

A FK pela interface do pgAdmin



Esquema simplificado da tela — a aparência muda um pouco entre versões do pgAdmin.

1. Em **Tables** → **matriculas** → **Constraints**: botão direito → **Create** → **Foreign Key…**
2. **General**: Name = `fk_mat_aluno`.
3. **Columns**: *Local column* (a FK) → *References* (a tabela pai) → *Referencing* (a PK) → **Add**.
4. **Action**: em *On delete*, deixe **NO ACTION** — o padrão, que recusa.
5. Confira a aba **SQL**: é o mesmo `ALTER TABLE … ADD CONSTRAINT`. Depois, **Save**.

Criando a tabela?

O mesmo formulário fica em **Create - Table** → **Constraints** → **Foreign Key** — a parte que deixamos para depois no Capítulo 3.

Criou por SQL? A FK aparece em **Constraints**. E o **ERD Tool** do pgAdmin (botão direito no banco → *ERD For Database*) desenha as tabelas e as ligações a partir das FKs: o seu MER, gerado pelo próprio banco.

CHECK: uma condição que todo valor cumpre

```
coluna tipo CHECK (condição)

CONSTRAINT nome CHECK (condição)
```

A condição é escrita como no `WHERE`: `>` `BETWEEN` `IN` `AND`… O banco testa cada linha inserida ou alterada.

> Se a condição envolve **duas colunas**, ela vai no fim da tabela.

REGRA DO NEGÓCIO	CHECK
A nota vai de 0 a 10	`CHECK (nota BETWEEN 0 AND 10)`
Faltas nunca são negativas	`CHECK (faltas >= 0)`
Carga horária maior que zero	`CHECK (carga_horaria > 0)`
Situação só pode ser uma destas	`CHECK (situacao IN ('ativo', 'trancado'))`
No máximo 3 renovações	`CHECK (renovacoes BETWEEN 0 AND 3)`
Devolução não antes da retirada	`CHECK (data_devolucao >= data_retirada)`
Atendimento das 7h às 19h	`CHECK (hora BETWEEN '07:00' AND '19:00')`

EXEMPLO EXPLICADO • CHECK

CHECK em ação na escola

```
ALTER TABLE matriculas
  ADD CONSTRAINT ck_nota CHECK (nota BETWEEN 0 AND 10);
ALTER TABLE matriculas
  ADD CONSTRAINT ck_faltas CHECK (faltas >= 0);
ALTER TABLE disciplinas
  ADD CHECK (carga_horaria > 0);    -- sem nome
```

TESTANDO

```
UPDATE matriculas SET nota = 15 WHERE cod_mat = 1;
```

```
new row for relation "matriculas" violates check constraint
"ck_nota"
DETAIL: Failing row contains (1, 101, BD, 2026-02-02, 15.00, 2).
```

```
UPDATE disciplinas SET carga_horaria = 0 WHERE sigla_disc = 'LOG';
```

```
violates check constraint "disciplinas_carga_horaria_check"
```

Com nome ou sem nome?

Sem `CONSTRAINT nome`, o PostgreSQL inventa um: `tabela_coluna_check`. Funciona — mas o seu nome é mais fácil de ler na mensagem e de apagar depois.

```
ALTER TABLE matriculas DROP CONSTRAINT ck_nota;
```

Os dados antigos também passam pelo teste

Se alguma linha já quebra a regra, o banco não cria o CHECK:

```
check constraint "ck_max" of relation "matriculas" is violated
by some row
```

NULL passa pelo CHECK e pela FK

Lembra? NULL é “não sei”. Uma condição com NULL não dá falso — então o banco deixa passar.

VALOR DE NOTA	CHECK (NOTA BETWEEN 0 AND 10)
`8.5`	ACEITO verdadeiro
`15`	RECUSADO falso
`NULL`	ACEITO “não sei” — não é falso

A nota da Júlia, ainda não lançada, continua podendo ficar vazia. É isso que queremos.

Na FK é igual

```
INSERT INTO disciplinas
  (sigla_disc, nome_disc, carga_horaria)
VALUES ('ROB', 'Robótica', 40);
```

```
INSERT 0 1   ← cod_prof NULL: disciplina ainda sem professor
```

Quer proibir o vazio? Junte `NOT NULL` à regra: `rm_aluno integer NOT NULL REFERENCES alunos`.

As seis restrições — e a palavra que aparece no erro

RESTRIÇÃO	GARANTE	NA ESCOLA	NO ERRO APARECE
`PRIMARY KEY`	identifica a linha: única e nunca vazia	rm_aluno	`duplicate key … _pkey`
`NOT NULL`	campo obrigatório	nome_aluno	`not-null constraint`
`UNIQUE`	não repete — mas não é a PK	cpf_aluno	`duplicate key … _key`
`DEFAULT`	valor automático quando nada é informado	faltas → 0	— não gera erro
`FOREIGN KEY`	só aceita códigos que existem no pai	matriculas.rm_aluno	`foreign key constraint`
`CHECK`	todo valor cumpre uma condição	nota de 0 a 10	`check constraint`

Quanto mais regras no banco, menos bobagem nos dados — não importa se quem grava é você, o pgAdmin ou o sistema que você vai programar.

PRÁTICA 3 • 15 MIN • INDIVIDUAL

A biblioteca ganha regras

No banco `biblioteca` do Capítulo 3. Dê nome a cada regra — a primeira pela interface do pgAdmin, as outras com `ALTER TABLE`.

1. Todo livro aponta para uma editora que existe — **pela interface**.
2. Todo empréstimo aponta para um livro e para um leitor que existem.
3. Empréstimo sem livro ou sem leitor? Não pode mais: torne as duas colunas obrigatórias.
4. A biblioteca permite no máximo **3 renovações**.
5. A data de devolução nunca é anterior à data de retirada.
6. Confira em **Constraints**, no pgAdmin, se as cinco regras aparecem com o nome que você deu.

Deu erro no ALTER?

Na Prática 5 do Capítulo 3, o empréstimo do leitor 99 foi aceito. Ele ainda está lá:

```
DETAIL: Key (cod_leitor)=(99) is not
present in table "leitores".
```

Corrija o dado — exclua ou aponte para um leitor que existe — e rode o ALTER de novo.

NOMES SUGERIDOS

`fk_livro_editora` · `fk_emp_livro`
`fk_emp_leitor` · `ck_renovacoes`
`ck_devolucao`

PENSE E RESPONDA Por que a FK do leitor só foi criada depois de corrigir o empréstimo do leitor 99?

PRÁTICA 4 • 8 MIN • INDIVIDUAL

Tente enganar o SGBD — parte 2

No banco `biblioteca`, com as regras da Prática 3. Antes de rodar, escreva o que espera; depois, a regra que barrou.

- **a** Um livro novo (código 30) da **editora 9**, que não existe.
- **b** Um empréstimo novo (código 30) do **livro 99**.
- **c** Excluir a **editora 1**, Aurora.
- **d** O empréstimo 2 com **4 renovações**.
- **e** Devolução do empréstimo 5 em `2026-09-01` — antes da retirada.
- **f** Um livro novo (código 31) **sem editora**.
- **g** Excluir a **leitora 3**, Lia Rocha.
- **h** O empréstimo 32: livro 12 para a Júlia (2), sem informar datas nem renovações.

PENSE E RESPONDA Só duas tentativas passaram. O banco estava certo em aceitar cada uma? O que mudaria para recusar a (f)?

SEÇÃO 03

Do MER ao script

03

Implementar um DER/MER inteiro num arquivo .sql — que monta o banco do zero quantas vezes for preciso.

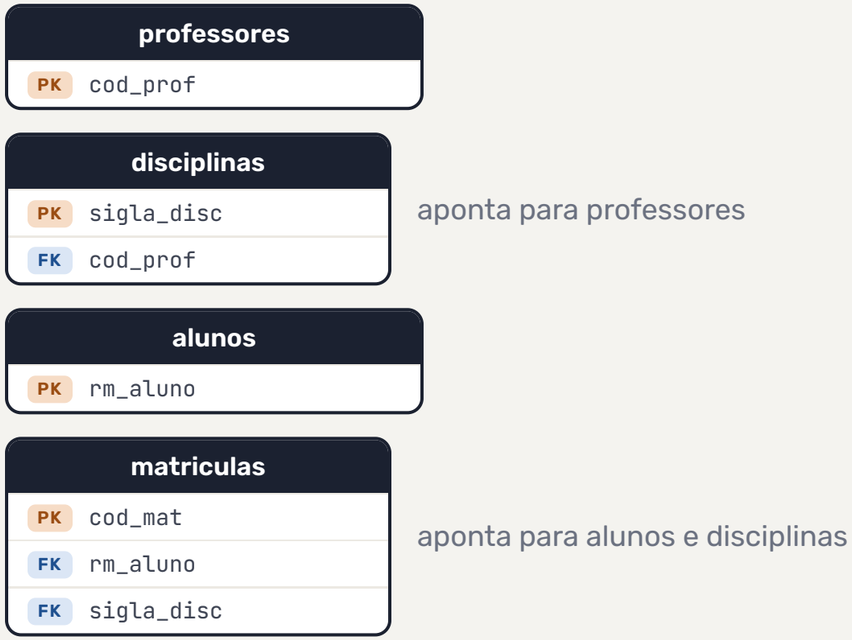
Cada peça do MER vira um pedaço do SQL

NO MER	NO SCRIPT	EXEMPLO
Entidade	`CREATE TABLE`	`CREATE TABLE alunos (…)`
Atributo	coluna + tipo	`nome_aluno varchar(100)`
PK chave primária	`PRIMARY KEY`	`rm_aluno integer PRIMARY KEY`
1:N • FK no lado N	`REFERENCES` na tabela do lado N	`cod_prof integer REFERENCES professores`
1:1 • FK em um dos lados	`REFERENCES` + `UNIQUE`	`chefe_depto integer UNIQUE REFERENCES …`
N:N • tabela nova	tabela com duas FKs + `UNIQUE` do par	`matriculas` com `rm_aluno` e `sigla_disc`
Regras do briefing	`NOT NULL` • `UNIQUE` • `DEFAULT` • `CHECK`	“o CPF não repete”, “a nota vai de 0 a 10”



No 1:1, o `UNIQUE` na FK é o que garante o “um”: o mesmo funcionário não chefia dois departamentos. Sem ele, vira 1:N.

A ordem do script importa



1 • APAGAR • DROP

Filhos antes dos pais

`matriculas → disciplinas → alunos → professores`. Quem tem FK sai primeiro, senão o DROP é recusado.

2 • CRIAR • CREATE

Pais antes dos filhos

`professores → alunos → disciplinas → matriculas`. O `REFERENCES` só funciona se a tabela pai já existe.

3 • DADOS • INSERT

Pais antes dos filhos de novo

Primeiro professores e alunos; as matrículas por último. A FK confere cada código.

Truque: no MER, comece pelas tabelas que **não têm FK**. Elas são as primeiras a criar e as últimas a apagar.

EXEMPLO EXPLICADO · ESCOLA.SQL · 1 DE 3

O script da escola: apagar e criar os pais

```sql
-- escola.sql · recria o banco da escola do zero

-- 1) Apagar: filhos antes dos pais
DROP TABLE IF EXISTS matriculas;
DROP TABLE IF EXISTS disciplinas;
DROP TABLE IF EXISTS alunos;
DROP TABLE IF EXISTS professores;

-- 2) Criar: pais antes dos filhos
CREATE TABLE professores (
  cod_prof   integer      PRIMARY KEY,
  nome_prof  varchar(100) NOT NULL,
  email_prof varchar(100) UNIQUE
);

CREATE TABLE alunos (
  rm_aluno     integer      PRIMARY KEY,
  nome_aluno   varchar(100) NOT NULL,
  cpf_aluno    char(11)     UNIQUE,
  data_nasc    date,
  cidade_aluno varchar(60)
);
```

Por que começar apagando?

Para o script rodar **quantas vezes for preciso**. Errou algo? Corrige o arquivo e roda tudo de novo.

Por que IF EXISTS?

Na primeira vez, as tabelas não existem. Sem ele, o primeiro DROP já daria erro e o script pararia.

Comentários

`--` não é executado. Use para separar as partes: quem abrir o arquivo depois agradece.

Os filhos: FK, CHECK e UNIQUE do par

```sql
CREATE TABLE disciplinas (
  sigla_disc    varchar(10) PRIMARY KEY,
  nome_disc     varchar(60) NOT NULL,
  carga_horaria integer     NOT NULL CHECK (carga_horaria > 0),
  cod_prof      integer     REFERENCES professores (cod_prof)
);

CREATE TABLE matriculas (
  cod_mat    integer      PRIMARY KEY,
  rm_aluno   integer      NOT NULL REFERENCES alunos (rm_aluno),
  sigla_disc varchar(10)  NOT NULL REFERENCES disciplinas (sigla_disc),
  data_mat   date         NOT NULL DEFAULT CURRENT_DATE,
  nota       numeric(4,2) CHECK (nota BETWEEN 0 AND 10),
  faltas     integer      NOT NULL DEFAULT 0 CHECK (faltas >= 0),
  UNIQUE (rm_aluno, sigla_disc)
);
```

cod_prof sem NOT NULL

Uma disciplina pode existir antes de ter professor. A FK confere o código quando ele vier.

rm_aluno e sigla_disc com NOT NULL

Matrícula sem aluno ou sem disciplina não faz sentido.

UNIQUE do par

Uma PK própria (`cod_mat`) e, mesmo assim, o aluno não entra duas vezes na mesma disciplina.

Os dados — com alguns casos novos

```sql
-- 3) Dados: pais antes dos filhos
INSERT INTO professores (cod_prof, nome_prof, email_prof) VALUES
  (7, 'Rui Lima',     'rui.lima@escola.edu.br'),
  (8, 'Ana Reis',     'ana.reis@escola.edu.br'),
  (9, 'Marcos Paulo', NULL);

INSERT INTO alunos (rm_aluno, nome_aluno, cpf_aluno, data_nasc, cidade_aluno) VALUES
  (101, 'André Silva', '11122233344', '2008-03-14', 'Recife'),
  (102, 'Carla Motta', '22233344455', '2007-11-02', 'Olinda'),
  (103, 'Bruno Lima',  '33344455566', '2009-06-25', 'Recife'),
  (104, 'Júlia Reis',  '44455566677', '2008-09-10', 'Jaboatão'),
  (105, 'Davi Rocha',  '55566677788', '2006-01-30', 'Recife'),
  (106, 'Lia Rocha',   '66677788899', '2009-04-18', 'Olinda');

INSERT INTO disciplinas (sigla_disc, nome_disc, carga_horaria, cod_prof) VALUES
  ('BD',  'Banco de Dados',          80, 7),
  ('LOG', 'Lógica de Programação',   60, 8),
  ('WEB', 'Desenvolvimento Web',     80, 7),
  ('SEG', 'Segurança da Informação', 40, NULL);

INSERT INTO matriculas (cod_mat, rm_aluno, sigla_disc, data_mat, nota, faltas) VALUES
  (1, 101, 'BD',  '2026-02-02', 8.5,  2),  (2, 101, 'LOG', '2026-02-03', 7.0,  0),
  (3, 102, 'BD',  '2026-02-02', 9.5,  1),  (4, 103, 'WEB', '2026-02-05', 6.0, 10),
  (5, 104, 'BD',  '2026-02-04', NULL, 0),  (6, 105, 'LOG', '2026-02-03', 5.5, 16);

SELECT * FROM matriculas ORDER BY cod_mat;   -- conferir: 6 linhas
```

OS CASOS NOVOS • VÃO APARECER NO JOIN

Marcos Paulo

Professor sem nenhuma disciplina.

Lia Rocha

Aluna cadastrada, sem matrícula.

Segurança da Informação

Disciplina sem professor e sem alunos.

Junte as três partes num arquivo **escola.sql**: ele substitui as tabelas da escola do Capítulo 3.

Rodando um script na Query Tool

1. Botão direito no banco **escola** → **Query Tool**. Confira o nome no topo.
2. Abra o arquivo (ícone de pasta, **Open File**) ou cole o conteúdo do `escola.sql`.
3. Sem selecionar nada, **Execute (F5)**: roda o arquivo inteiro, de cima para baixo.
4. A aba **Data Output** mostra só o resultado do **último** SELECT. A aba **Messages** mostra o resto.
5. Refresh em **Tables**. Clique numa tabela → **Constraints**: as FKs e os CHECKs estão lá.
6. Mudou o script? Salve (**Save File**) — ele é parte da entrega.

NA PRIMEIRA VEZ, A ABA MESSAGES MOSTRA

```
NOTICE: table "matriculas" does not exist, skipping
NOTICE: table "disciplinas" does not exist, skipping
…
```

É só um aviso do `IF EXISTS`, não um erro. Da segunda vez em diante, ele some.

Deu erro no meio?

O script para no comando com erro — o que veio antes já rodou. Corrija o arquivo e execute **tudo** de novo: o DROP do começo limpa a bagunça.

EXEMPLO EXPLICADO • 1:1

Quando duas tabelas apontam uma para a outra

O departamento tem funcionários (1:N) e um chefe (1:1) — o caso do Capítulo 1. Cada tabela tem uma FK para a outra: quem criar primeiro?

```
CREATE TABLE departamentos (
  cod_depto   integer     PRIMARY KEY,
  nome_depto  varchar(60) NOT NULL,
  chefe_depto integer     UNIQUE        -- FK vem depois
);
CREATE TABLE funcionarios (
  cod_func  integer      PRIMARY KEY,
  nome_func varchar(100) NOT NULL,
  cod_depto integer      NOT NULL REFERENCES departamentos (cod_depto)
);
-- agora as duas existem: a FK que faltava, com ALTER
ALTER TABLE departamentos
  ADD CONSTRAINT fk_depto_chefe
  FOREIGN KEY (chefe_depto) REFERENCES funcionarios (cod_func);
```

UNIQUE = o “1” do 1:1

Ana já chefia Vendas. Pôr Ana como chefe de TI também:

```
duplicate key value violates unique
constraint
"departamentos_chefe_depto_key"
```

E para apagar?

Uma depende da outra: apague as duas no mesmo comando.

```
DROP TABLE IF EXISTS
  departamentos, funcionarios;
```

PRÁTICA 5 • 8 MIN • INDIVIDUAL

Planeje o script da oficina

O MER da **Oficina Turbo**, do Capítulo 2. Antes de escrever qualquer CREATE, planeje o script.

clientes	
PK	cod_cliente
	nome_cliente
	cpf_cliente
	telefone

veiculos	
PK	cod_veiculo
	placa
	modelo
FK	cod_cliente

mecanicos	
PK	cod_mecanico
	nome_mecanico

ordens_servico	
PK	cod_os
	data_os
	descricao
FK	cod_veiculo
FK	cod_mecanico

pecas	
PK	cod_peca
	nome_peca
	preco

itens_os	
PK	cod_item
	qtd
FK	cod_os
FK	cod_peca

1. Em que ordem vão os **CREATE TABLE**?
2. E os **DROP TABLE IF EXISTS**, no começo do script?
3. Escreva a linha de cada uma das **5 FKs**, com nome e `REFERENCES`. Toda FK aqui é obrigatória.
4. A quantidade de cada peça é **maior que zero**: escreva o CHECK.

FORMATO DA RESPOSTA

```
cod_cliente integer NOT NULL
  CONSTRAINT fk_veic_cliente
  REFERENCES clientes (cod_cliente)
```

PENSE E RESPONDA Três tabelas não têm FK. Elas podem ser criadas em qualquer ordem entre si? Por quê?

PRÁTICA 6 • 30 MIN • INDIVIDUAL

Implemente a Clínica Vida por script

O MER DO CAPÍTULO 2



REGRAS DA CLÍNICA

- Nome do convênio obrigatório e sem repetir.
- CRM obrigatório e sem repetir; nome e especialidade obrigatórios.
- Paciente: nome obrigatório, CPF sem repetir e sempre um convênio.
- Consulta: data e hora obrigatórias, das **7h às 19h** (tipo `time`); diagnóstico fica vazio até o atendimento.
- Um médico não atende duas consultas no **mesmo dia e hora**.

O QUE ENTREGAR • CLINICA.SQL

1. `CREATE DATABASE clinica` • 2. DROP + CREATE na ordem certa • 3. os dados do próximo slide • 4. um teste recusado para cada regra • 5. o pedido extra:

“Quero guardar o valor de cada consulta: nunca negativo, e 0 se não for informado.” — com `ALTER TABLE`.

PRÁTICA 6 • OS DADOS

Os dados da Clínica Vida

CONVENIOS

cod_conv	nome_conv	fone_conv
1	Saúde Total	(85) 3333-1000
2	Vida Mais	(85) 3333-2000
3	Bem Estar	(85) 3333-3000

MEDICOS

cod_med	crm_med	nome_med	especialidade_med
1	12345	Helena Duarte	Cardiologia
2	23456	Otávio Nunes	Clínica Geral
3	34567	Sofia Brandão	Dermatologia
4	45678	Caio Ferraz	Ortopedia

PACIENTES

cod_pac	nome_pac	cpf_pac	fone_pac	cod_conv
1	Marina Alves	10120230344	(85) 98888-0001	1
2	João Pedro Lins	20230340455	(85) 98888-0002	2
3	Beatriz Sales	30340450566	(85) 98888-0003	1
4	Renato Dias	40450560677	(85) 98888-0004	2
5	Clara Nogueira	50560670788	(85) 98888-0005	1

CONSULTAS

cod	data_cons	hora	diagnostico_cons	pac	med
1	2026-10-05	08:00	Hipertensão leve	1	1
2	2026-10-05	09:00	Virose	2	2
3	2026-10-06	14:00	Dermatite	3	3
4	2026-10-07	10:00	Alergia de contato	1	3
5	2026-10-27	08:00	*[null]*	4	1
6	2026-10-27	15:00	*[null]*	2	1
7	2026-10-28	09:30	*[null]*	4	3

`pac = cod_pac · med = cod_med · hora = hora_cons`

Testes que devem ser recusados

Paciente do convênio 9 • a Dra. Helena com outra consulta em 27/10 às 08:00 • consulta às 22:00 • excluir o convênio Saúde Total • consulta com valor –50.

PENSE Por que os convênios entram antes dos pacientes?

SEÇÃO 04

JOIN

04

A normalização separou os dados em várias tabelas. O JOIN junta de volta, só na hora da consulta.

A coordenação pediu o boletim

“Quero o nome do aluno, o nome da disciplina e a nota.” Cada pedaço está numa tabela diferente.

MATRICULAS

cod_mat	rm_aluno	sigla_disc	nota
1	101	BD	8.50
2	101	LOG	7.00
3	102	BD	9.50

… 6 linhas

ALUNOS

rm_aluno	nome_aluno
101	André Silva
102	Carla Motta
103	Bruno Lima

… 6 linhas

DISCIPLINAS

sigla_disc	nome_disc
BD	Banco de Dados
LOG	Lógica de Programação
WEB	Desenvolvimento Web

… 4 linhas

→

O QUE A COORDENAÇÃO QUER VER

nome_aluno	nome_disc	nota
André Silva	Banco de Dados	8.50

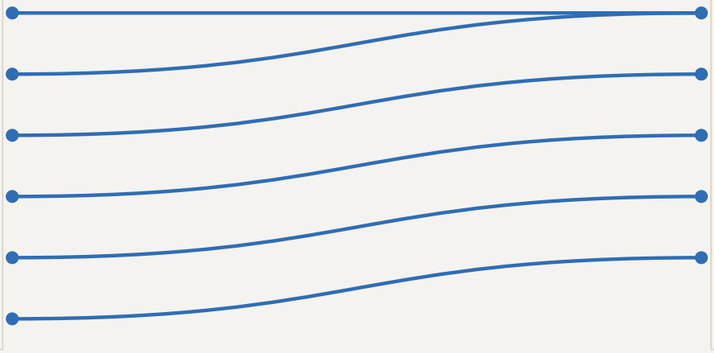
A matrícula guarda só os **códigos** — `101` e `BD`. Os nomes estão nas tabelas de origem. É exatamente isso que a normalização do Capítulo 2 fez: **cada fato num lugar só**.

Como o banco pensa um JOIN

Para cada linha de `matriculas`, ele procura em `alunos` a linha cujo `rm_aluno` é igual — e cola as duas.

MATRICULAS

cod_mat	rm_aluno	sigla_disc	nota
1	101	BD	8.50
2	101	LOG	7.00
3	102	BD	9.50
4	103	WEB	6.00
5	104	BD	[null]
6	105	LOG	5.50



ALUNOS

rm_aluno	nome_aluno	
101	André Silva	
102	Carla Motta	
103	Bruno Lima	
104	Júlia Reis	
105	Davi Rocha	
106	Lia Rocha	sem par

```
ON alunos.rm_aluno = matriculas.rm_aluno
```

FK = PK: a ligação do JOIN é a mesma seta do MER. O `ON` diz ao banco qual coluna da filha casa com qual coluna do pai.

O André aparece **duas vezes** no resultado — uma por matrícula. A Lia, sem matrícula, **não aparece**.

INNER JOIN: o molde

```
SELECT colunas
FROM tabela_a
[INNER] JOIN tabela_b
  ON tabela_b.coluna = tabela_a.coluna
[WHERE condição]
[ORDER BY coluna];
```

MAIÚSCULA escreva igual *nome* troque pelo seu […] opcional

- `JOIN` sozinho já é `INNER JOIN`: o INNER é opcional.
- `ON` é obrigatório: é a condição que liga as linhas — quase sempre **FK = PK**.
- O resultado traz só as linhas que **encontraram par** nas duas tabelas.
- `WHERE` e `ORDER BY` continuam valendo — depois dos JOINs.
- Cada JOIN a mais traz mais uma tabela: um JOIN para cada seta do MER que você percorre.

O primeiro JOIN

```sql
SELECT matriculas.cod_mat,
       alunos.nome_aluno,
       matriculas.sigla_disc,
       matriculas.nota
FROM matriculas
JOIN alunos
  ON alunos.rm_aluno = matriculas.rm_aluno;
```

cod_mat	nome_aluno	sigla_disc	nota
1	André Silva	BD	8.50
2	André Silva	LOG	7.00
3	Carla Motta	BD	9.50
4	Bruno Lima	WEB	6.00
5	Júlia Reis	BD	*[null]*
6	Davi Rocha	LOG	5.50

6 linhas — uma por matrícula

`alunos.nome_aluno` = a coluna `nome_aluno` **da tabela** `alunos`. É o nome completo da coluna.

Coluna ambígua e apelidos de tabela

SEM DIZER DE QUAL TABELA

```
SELECT rm_aluno, nome_aluno, sigla_disc
FROM matriculas
JOIN alunos ON alunos.rm_aluno = matriculas.rm_aluno;
```

```
ERROR: column reference "rm_aluno" is ambiguous
```

`rm_aluno` existe nas **duas** tabelas. O banco não adivinha de qual você quer.

COM APELIDOS

```
SELECT m.rm_aluno, a.nome_aluno, m.sigla_disc
FROM matriculas m
JOIN alunos a ON a.rm_aluno = m.rm_aluno;
```

```
6 linhas
```

O apelido vem logo depois do nome da tabela. Daí em diante, `m.` é `matriculas.` e `a.` é `alunos.`

Regra da turma: em consultas com JOIN, use apelido em **todas** as colunas — mesmo nas que não são ambíguas. Fica claro de onde vem cada dado.

EXEMPLO EXPLICADO • TRÊS TABELAS

O boletim completo

```sql
SELECT a.nome_aluno, d.nome_disc, m.nota
FROM matriculas m
JOIN alunos      a ON a.rm_aluno   = m.rm_aluno
JOIN disciplinas d ON d.sigla_disc = m.sigla_disc
ORDER BY a.nome_aluno, d.nome_disc;
```

1. Comece pela tabela do **meio**: `matriculas` tem as duas FKs.
2. Um JOIN para cada FK: `rm_aluno` leva a `alunos`, `sigla_disc` leva a `disciplinas`.
3. No SELECT, escolha as colunas de qualquer uma das três.

nome_aluno	nome_disc	nota
André Silva	Banco de Dados	8.50
André Silva	Lógica de Programação	7.00
Bruno Lima	Desenvolvimento Web	6.00
Carla Motta	Banco de Dados	9.50
Davi Rocha	Lógica de Programação	5.50
Júlia Reis	Banco de Dados	*[null]*

6 linhas · nenhum código, só nomes

EXEMPLO EXPLICADO • QUATRO TABELAS

Siga o caminho das FKs

“Quem é o professor de cada aluno, em cada disciplina?” O professor está a **duas setas** de distância da matrícula.



3 setas no caminho = 3 JOINs

```sql
SELECT a.nome_aluno, d.nome_disc, p.nome_prof
FROM matriculas m
JOIN alunos      a ON a.rm_aluno   = m.rm_aluno
JOIN disciplinas d ON d.sigla_disc = m.sigla_disc
JOIN professores p ON p.cod_prof   = d.cod_prof
ORDER BY p.nome_prof, a.nome_aluno;
```

nome_aluno	nome_disc	nome_prof
André Silva	Lógica de Programação	Ana Reis
Davi Rocha	Lógica de Programação	Ana Reis
André Silva	Banco de Dados	Rui Lima
Bruno Lima	Desenvolvimento Web	Rui Lima
Carla Motta	Banco de Dados	Rui Lima
Júlia Reis	Banco de Dados	Rui Lima

Repare no último JOIN: ele liga `professores` a `d`, não a `m`. A FK `cod_prof` mora em `disciplinas`.

JOIN com WHERE e ORDER BY

"O boletim só dos alunos de Recife."

```
SELECT a.nome_aluno, d.nome_disc, m.nota
FROM matriculas m
JOIN alunos      a ON a.rm_aluno   = m.rm_aluno
JOIN disciplinas d ON d.sigla_disc = m.sigla_disc
WHERE a.cidade_aluno = 'Recife'
ORDER BY a.nome_aluno, d.nome_disc;
```

nome_aluno	nome_disc	nota
André Silva	Banco de Dados	8.50
André Silva	Lógica de Programação	7.00
Bruno Lima	Desenvolvimento Web	6.00
Davi Rocha	Lógica de Programação	5.50

A ORDEM QUE VOCÊ ESCREVE

`SELECT → FROM → JOIN … ON → WHERE → ORDER BY → LIMIT`

A ORDEM EM QUE O BANCO EXECUTA

`FROM + JOINs → WHERE → SELECT → ORDER BY → LIMIT`

Primeiro o banco monta a "tabelona" juntada. Só depois filtra, escolhe as colunas e ordena.

Por isso o `WHERE` pode filtrar por uma coluna de **qualquer** tabela do JOIN — até uma que nem aparece no SELECT.

Quem ficou de fora do INNER JOIN?

O INNER JOIN só mostra linhas que **encontraram par**. Três casos do `escola.sql` nunca aparecem:

ALUNA SEM MATRÍCULA

Lia Rocha • 106

Nenhuma linha de `matriculas` aponta para ela.

DISCIPLINA SEM PROFESSOR

Segurança da Informação

`cod_prof` é NULL: não há par em `professores`.

PROFESSOR SEM DISCIPLINA

Marcos Paulo • 9

Nenhuma disciplina tem `cod_prof = 9`.

```sql
SELECT d.nome_disc, p.nome_prof
FROM disciplinas d
JOIN professores p ON p.cod_prof = d.cod_prof;
```

nome_disc	nome_prof
Banco de Dados	Rui Lima
Lógica de Programação	Ana Reis
Desenvolvimento Web	Rui Lima

3 linhas — são 4 disciplinas

Quando a pergunta diz **“todos”** ou **“quem não tem”**, o INNER JOIN não basta.

LEFT JOIN: todos da tabela da esquerda

```sql
SELECT a.nome_aluno, m.sigla_disc, m.nota
FROM alunos a                -- esquerda
LEFT JOIN matriculas m       -- direita
  ON m.rm_aluno = a.rm_aluno
ORDER BY a.nome_aluno;
```

- **Esquerda** é a tabela do `FROM`; **direita**, a do `LEFT JOIN`.
- Toda linha da esquerda aparece — com par ou sem par.
- Sem par? As colunas da direita vêm `NULL`.

nome_aluno	sigla_disc	nota
André Silva	BD	8.50
André Silva	LOG	7.00
Bruno Lima	WEB	6.00
Carla Motta	BD	9.50
Davi Rocha	LOG	5.50
Júlia Reis	BD	*[null]*
Lia Rocha	*[null]*	*[null]*

7 linhas · a Lia aparece, sem matrícula

A ordem das tabelas importa: `matriculas LEFT JOIN alunos` daria outra resposta.

EXEMPLO EXPLICADO · LEFT JOIN + IS NULL

“Quem não tem?”

LEFT JOIN traz todos; `WHERE … IS NULL` fica só com quem não encontrou par. Teste a **PK** da tabela da direita.

Alunos sem matrícula

```sql
SELECT a.rm_aluno, a.nome_aluno
FROM alunos a
LEFT JOIN matriculas m
  ON m.rm_aluno = a.rm_aluno
WHERE m.cod_mat IS NULL;
```

rm_aluno	nome_aluno
106	Lia Rocha

Disciplinas sem alunos

```sql
SELECT d.sigla_disc, d.nome_disc
FROM disciplinas d
LEFT JOIN matriculas m
  ON m.sigla_disc = d.sigla_disc
WHERE m.cod_mat IS NULL;
```

sigla_disc	nome_disc
SEG	Segurança da Informação

Professores sem disciplina

```sql
SELECT p.nome_prof
FROM professores p
LEFT JOIN disciplinas d
  ON d.cod_prof = p.cod_prof
WHERE d.sigla_disc IS NULL;
```

nome_prof
Marcos Paulo

Molde: `FROM quem_eu_quero LEFT JOIN onde_procuro ON … WHERE pk_de_onde_procuro IS NULL`

INNER × LEFT, lado a lado

INNER JOIN • SÓ QUEM TEM PAR

```sql
SELECT d.nome_disc, p.nome_prof
FROM disciplinas d
JOIN professores p ON p.cod_prof = d.cod_prof;
```

nome_disc	nome_prof
Banco de Dados	Rui Lima
Lógica de Programação	Ana Reis
Desenvolvimento Web	Rui Lima

3 linhas

LEFT JOIN • TODAS AS DISCIPLINAS

```sql
SELECT d.nome_disc, p.nome_prof
FROM disciplinas d
LEFT JOIN professores p ON p.cod_prof = d.cod_prof;
```

nome_disc	nome_prof
Banco de Dados	Rui Lima
Lógica de Programação	Ana Reis
Desenvolvimento Web	Rui Lima
Segurança da Informação	*[null]*

4 linhas

TIPO	MOSTRA	NA PRÁTICA
`JOIN` • `INNER JOIN`	só as linhas com par dos dois lados	o mais usado: boletins, relatórios, listagens
`LEFT JOIN`	tudo da esquerda + o par, se houver	“todos os…”, “quem não tem…”
`RIGHT JOIN`	o espelho: tudo da direita	raro — troque a ordem das tabelas e use LEFT

EXEMPLO EXPLICADO • ESTRATÉGIA

Da pergunta ao JOIN

PERGUNTA “Quais alunos têm aula com o professor Rui Lima?”

1 O que mostrar?

O nome do aluno → `alunos`.

2 Por onde filtrar?

O nome do professor → `professores`.

3 Qual o caminho?

No MER: alunos ← matriculas → disciplinas → professores. **3 JOINs**.

4 Repete?

O André tem duas matrículas com o Rui? Use `DISTINCT`.

```sql
SELECT DISTINCT a.nome_aluno
FROM matriculas m
JOIN alunos      a ON a.rm_aluno   = m.rm_aluno
JOIN disciplinas d ON d.sigla_disc = m.sigla_disc
JOIN professores p ON p.cod_prof   = d.cod_prof
WHERE p.nome_prof = 'Rui Lima'
ORDER BY a.nome_aluno;
```

nome_aluno
André Silva
Bruno Lima
Carla Motta
Júlia Reis

4 linhas

EXEMPLO EXPLICADO · JOIN + CÁLCULO

Alunos em risco

"Quem está com nota abaixo de 7 ou passou do limite de faltas?" O limite é 25% da carga horária — que está em `disciplinas`.

```
SELECT a.nome_aluno, d.nome_disc, m.nota, m.faltas,
       d.carga_horaria * 0.25 AS limite_faltas
FROM matriculas m
JOIN alunos      a ON a.rm_aluno   = m.rm_aluno
JOIN disciplinas d ON d.sigla_disc = m.sigla_disc
WHERE m.nota < 7
   OR m.faltas > d.carga_horaria * 0.25;
```

nome_aluno	nota	faltas	limite
Bruno Lima	6.00	10	20.00
Davi Rocha	5.50	16	15.00

2 linhas · coluna nome_disc omitida aqui

O Bruno entra pela nota. O Davi, pelas duas regras. Sem o JOIN, a carga horária não estaria disponível para comparar.

Por que a Júlia não aparece? A nota dela é NULL — e `NULL < 7` não é verdadeiro.

04 • JOIN • ATENÇÃO

Erros comuns com JOIN

ERRO	O QUE ACONTECE	COMO EVITAR
Esquecer o `ON`	`syntax error at or near ";"`	Todo JOIN leva o seu ON.
Coluna sem apelido existente nas duas	`column reference "rm_aluno" is ambiguous`	Apelido em todas as colunas.
ON ligando as colunas erradas `ON a.rm_aluno = m.cod_mat`	**Nenhum erro.** Volta vazio — ou volta errado.	Confira no MER: o ON liga a **FK** à **PK** que ela aponta.
Tabelas com vírgula, sem ligação `FROM alunos a, matriculas m`	**Nenhum erro.** 6 alunos × 6 matrículas = **36 linhas**: cada aluno com cada matrícula.	Use sempre `JOIN … ON`, nunca a vírgula.
INNER quando a pergunta diz “todos”	**Nenhum erro.** A Lia some do relatório.	“Todos” e “quem não tem” pedem `LEFT JOIN`.

⚠ Os três últimos são os mais perigosos: o banco não reclama. Antes de confiar no resultado, pergunte: **quantas linhas eu esperava?**

PRÁTICA 7 • 8 MIN • INDIVIDUAL

Leia o JOIN

Banco `escola`, depois do `escola.sql`. Sem executar: **quantas linhas** volta cada consulta, e quais? Depois, rode e confira.

a

```sql
SELECT d.nome_disc, p.nome_prof
FROM disciplinas d
JOIN professores p
  ON p.cod_prof = d.cod_prof;
```

b

```sql
SELECT p.nome_prof, d.nome_disc
FROM professores p
LEFT JOIN disciplinas d
  ON d.cod_prof = p.cod_prof;
```

c

```sql
SELECT a.nome_aluno
FROM alunos a
JOIN matriculas m
  ON m.rm_aluno = a.rm_aluno
WHERE m.sigla_disc = 'BD';
```

d

```sql
SELECT a.nome_aluno, m.sigla_disc
FROM alunos a
LEFT JOIN matriculas m
  ON m.rm_aluno = a.rm_aluno;
```

e

```sql
SELECT a.nome_aluno
FROM alunos a
LEFT JOIN matriculas m
  ON m.rm_aluno = a.rm_aluno
WHERE m.cod_mat IS NULL;
```

f

```sql
SELECT a.nome_aluno, m.sigla_disc
FROM alunos a, matriculas m;
```

PENSE E RESPONDA Em (b), por que o Rui Lima aparece duas vezes? E o Marcos Paulo, como aparece?

PRÁTICA 8 • 10 MIN • INDIVIDUAL

Encontre o erro

No banco `clinica` da Prática 6. Rode cada comando, leia a mensagem e corrija.

a

```
CREATE TABLE teste (cod integer PRIMARY KEY,
  nome varchar(50) NOT NULL,);
```

b

```
CREATE TABLE exames (cod_exame integer PRIMARY KEY,
  cod_pac integer REFERENCES paciente (cod_pac));
```

c

```
SELECT nome_pac, nome_conv
FROM pacientes JOIN convenios;
```

d

```
SELECT cod_conv, nome_pac FROM pacientes p
JOIN convenios c ON c.cod_conv = p.cod_conv;
```

e

```
ALTER TABLE medicos
  ADD COLUMN email_med varchar(100) NOT NULL;
```

f

```
DROP TABLE convenios;
```

g

```
SELECT p.nome_pac, m.nome_med FROM consultas co
JOIN pacientes p ON p.cod_pac = co.cod_med
JOIN medicos   m ON m.cod_med = co.cod_med;
```

h

```
SELECT p.nome_pac, c.nome_conv
FROM pacientes p, convenios c;
```

PENSE E RESPONDA Dois comandos não dão erro nenhum — e mesmo assim estão errados. Quais? Como você percebeu?

PRÁTICA 9 • 25 MIN • INDIVIDUAL

As perguntas da Clínica Vida

No banco `clinica`. Um SELECT para cada pergunta, salvo em `clinica.sql` com um comentário: `-- Prática 9 · 1`.

1. Os pacientes com o nome do convênio de cada um, em ordem alfabética.
2. A agenda: data, hora, paciente e médico, em ordem de data e hora.
3. As consultas da Dra. Helena Duarte: data, hora e paciente.
4. As consultas dos pacientes do convênio Saúde Total, com o médico. *Dica: 4 tabelas.*
5. As consultas ainda sem diagnóstico, com paciente e médico.
6. Os médicos que ainda não têm nenhuma consulta.
7. Os pacientes que nunca consultaram.
8. Os convênios sem nenhum paciente.

DESAFIO Os pacientes atendidos pela Dermatologia, sem repetir nomes. Quantos são?

04 • CONSULTA RÁPIDA

Cola de sintaxe do capítulo

CRIAR

```
CREATE TABLE tabela (
  col tipo PRIMARY KEY,
  col tipo NOT NULL,
  col tipo UNIQUE,
  col tipo DEFAULT valor,
  col tipo REFERENCES pai (pk),
  col tipo CHECK (condição)
);
```

ALTERAR

```
ALTER TABLE tabela ação;
-- ações:
ADD COLUMN col tipo
DROP COLUMN col
RENAME COLUMN a TO b
ALTER COLUMN col TYPE tipo
ALTER COLUMN col SET NOT NULL
ADD CONSTRAINT nome regra
DROP CONSTRAINT nome
```

APAGAR

```
DROP TABLE IF EXISTS tabela;
DROP DATABASE banco;

-- script:
-- DROP:   filhos → pais
-- CREATE: pais → filhos
-- INSERT: pais → filhos
```

FK NUMA TABELA PRONTA

```
ALTER TABLE filha
  ADD CONSTRAINT fk_nome
  FOREIGN KEY (col)
  REFERENCES pai (pk)
  [ON DELETE …];
```

JOIN

```
SELECT a.col, b.col
FROM a
JOIN b ON b.pk = a.fk
WHERE …
ORDER BY …;
```

QUEM NÃO TEM

```
SELECT a.col
FROM a
LEFT JOIN b ON b.fk = a.pk
WHERE b.pk IS NULL;
```

04 • CHECKLIST

Confira antes de chamar o professor

DDL

- [ ] Prática 1 e 2: eventos criada, alterada e apagada só com SQL.
- [ ] Sei a diferença entre DELETE, DROP COLUMN e DROP TABLE.
- [ ] Toda Query Tool aberta no banco certo.

RESTRIÇÕES E SCRIPT

- [ ] Escola com 3 FKs e os CHECKs de nota, faltas e carga.
- [ ] Biblioteca com FK e CHECK, órfãos corrigidos.
- [ ] Cada FK com nome, mesmo tipo da PK e NOT NULL quando obrigatória.
- [ ] `clinica.sql` roda do zero sem erro — duas vezes seguidas.
- [ ] Um teste recusado para cada regra.

JOIN

- [ ] Todo JOIN com ON ligando FK = PK.
- [ ] Apelido em todas as colunas.
- [ ] “Todos” e “quem não tem” com LEFT JOIN.
- [ ] Contei as linhas: o resultado bate com o esperado?

Resumo

DDL
CREATE · ALTER · DROP
define a estrutura
script .sql repetível

CREATE TABLE
coluna tipo regras,
última sem vírgula
pais antes dos filhos

ALTER TABLE
ADD · DROP COLUMN
RENAME · TYPE
SET NOT NULL · DEFAULT

DROP
leva estrutura e dados
IF EXISTS no script
filhos antes dos pais

FOREIGN KEY
REFERENCES pai (pk)
só aceita o que existe
pai não sai com filhos

CHECK
CHECK (condição)
BETWEEN · IN · >=
NULL passa

Do MER ao SQL
1:N → FK no lado N
1:1 → FK + UNIQUE
N:N → tabela + 2 FKs

Restrição com nome
CONSTRAINT ck_nota …
erro legível
DROP CONSTRAINT fácil

INNER JOIN
JOIN b ON b.pk = a.fk
só quem tem par
um JOIN por seta

Apelidos
FROM matriculas m
m.nota · a.nome_aluno
fim do "ambiguous"

LEFT JOIN
todos da esquerda
sem par → NULL
+ IS NULL = quem não tem

Ordem
FROM + JOIN → WHERE
→ SELECT → ORDER BY
→ LIMIT

Armadilhas: Query Tool no banco errado • vírgula depois da última coluna • NOT NULL em coluna nova de tabela cheia • DROP sem pensar nas FKs • JOIN sem ON • coluna ambígua • INNER quando a pergunta diz “todos” • vírgula entre tabelas no FROM

ATIVIDADE DE CONSOLIDAÇÃO • INDIVIDUAL

Do briefing ao JOIN

Academia Pulso: modelagem, script, mudanças com ALTER e consultas com JOIN — tudo junto, numa situação nova.

1 Modelar
DER e MER no draw.io, a partir do briefing
`30 min`

→

2 Implementar
o script academia.sql: DDL, regras e dados
`40 min`

→

3 Alterar
os pedidos novos da dona, com ALTER
`20 min`

→

4 Consultar
as perguntas da dona, com JOIN
`40 min`

Usa os Capítulos 1 a 4: entidades, cardinalidade, DER, MER, formas normais, tipos, restrições, CREATE, ALTER, INSERT, SELECT e JOIN.

CONSOLIDAÇÃO • O BRIEFING

Academia Pulso

MENSAGEM DA DONA DA ACADEMIA

“Aqui na Pulso, tudo ainda é anotado em fichas de papel. Quero um sistema simples.

Cada **aluno** assina **um plano**: Mensal, Trimestral ou Anual. Do plano, guardo o nome — que não se repete —, o valor por mês e a duração em meses. Do aluno: nome, CPF (não pode repetir), telefone, data de nascimento — nem todo mundo informa —, a data da matrícula — se ninguém disser, é hoje — e a situação: *ativo* ou *trancado*. Todo aluno começa ativo.

Temos **instrutores**: nome, especialidade e CREF, o registro profissional, que é único.

As **aulas coletivas** têm nome, dia da semana, horário, sala, capacidade — no máximo 30 alunos — e **um** instrutor responsável. O aluno se **inscreve** nas aulas que quiser, e guardo a data da inscrição. Ninguém se inscreve duas vezes na mesma aula.”

REGRAS DO PROJETO

- Nomes em minúsculas, com `_`, sem acento.
- Uma PK própria em cada tabela: `cod_…`
- DER sem atributos; MER com PK e FK.
- Toda ligação do MER vira **FOREIGN KEY**.
- Toda regra do briefing vira restrição: NOT NULL, UNIQUE, DEFAULT ou **CHECK**.
- Tudo num script que roda do zero: `academia.sql`.

CONSOLIDAÇÃO • ETAPA 1 DE 4 • MODELAGEM • 30 MIN

Modele a academia: DER e MER

1. Liste as entidades e os atributos de cada uma: os substantivos do briefing.
2. Encontre os relacionamentos — os verbos — e faça as duas perguntas em cada um: 1:1, 1:N ou N:N?
3. DER no draw.io: retângulos, losangos com verbo e 1/N nas pontas. Sem atributos.
4. MER: uma tabela por entidade, com todos os atributos, PK própria e FK em cada ligação.
5. O N:N vira uma tabela nova, com PK própria e o atributo do relacionamento.
6. Ao lado de cada tabela do MER, anote as regras: o que é obrigatório, único, padrão ou CHECK.

PISTAS NO BRIEFING

“assina **um** plano”	→ cardinalidade?
“**um** instrutor responsável”	→ onde fica a FK?
“se inscreve nas aulas que quiser”	→ que tipo de relacionamento?
“guardo a data da inscrição”	→ atributo de quem?
“no máximo 30”	→ que restrição?
“ativo ou trancado”	→ que restrição?
“duas vezes na mesma aula”	→ UNIQUE de quê?

PENSE E RESPONDA Quantas tabelas o seu MER tem? E quantas entidades o seu DER tem? Por que os números são diferentes?

CONSOLIDAÇÃO • ETAPA 2 DE 4 • IMPLEMENTAÇÃO • 40 MIN

Escreva o academia.sql

1. `CREATE DATABASE academia;` e uma Query Tool nova, **no banco academia**.
2. No arquivo: `DROP TABLE IF EXISTS` de todas as tabelas — filhos antes dos pais.
3. `CREATE TABLE` de cada tabela do seu MER — pais antes dos filhos — com tipos, PK, FK, NOT NULL, UNIQUE, DEFAULT e CHECK.
4. `INSERT` dos dados dos próximos dois slides, convertidos para o formato do banco.
5. Rode o arquivo inteiro com F5. Depois, rode **de novo**: tem que funcionar das duas vezes.
6. Teste o DEFAULT: cadastre um aluno informando só código, nome, CPF e plano. Vieram a data de hoje e "ativo"? Depois, exclua esse aluno de teste.

DECISÕES DE TIPO

O DADO	SUGESTÃO
Valor por mês	`numeric(6,2)`
Horário da aula	`time` → `'07:00'`
Dia da semana	`char(3)` → `'seg'`
CPF	`char(11)`
CREF	`varchar(20)`
Situação	`varchar(10)` + CHECK

Dias da semana: `seg ter qua qui sex sab` — sempre assim, sem acento e em minúsculas. Um CHECK garante.

As fichas da academia

Do jeito que a dona anotou. Você converte para o formato do banco: datas `'AAAA-MM-DD'`, valores com ponto.

ALUNOS

nº	aluno	CPF	telefone	nascimento	matrícula	situação	plano
1	Ana Beatriz Costa	10010010001	(85) 99100-0001	12/04/1999	10/01/2026	ativo	3
2	Bruno Teixeira	20020020002	(85) 99100-0002	30/08/2001	03/02/2026	ativo	1
3	Camila Freitas	30030030003	(85) 99100-0003	05/12/1995	15/03/2026	ativo	2
4	Diego Martins	40040040004	(85) 99100-0004	20/02/2003	02/05/2026	trancado	1
5	Elisa Moura	50050050005	(85) 99100-0005	07/07/1988	18/06/2026	ativo	3
6	Fábio Gomes	60060060006	(85) 99100-0006	*não informou*	01/08/2026	ativo	2
7	Gabriela Pinto	70070070007	(85) 99100-0007	25/10/2000	09/09/2026	ativo	1
8	Heitor Barros	80080080008	(85) 99100-0008	03/03/1992	01/10/2026	ativo	3

PLANOS

nº	plano	por mês	duração
1	Mensal	R$ 129,90	1 mês
2	Trimestral	R$ 109,90	3 meses
3	Anual	R$ 89,90	12 meses

INSTRUTORES

nº	instrutor	CREF	especialidade
1	Paula Rios	001234-G/CE	Spinning
2	Igor Matos	002345-G/CE	Treino funcional
3	Renata Luz	003456-G/CE	Pilates
4	Hugo Santana	004567-G/CE	Musculação

o nº de cada lista é o código: use como PK

A grade de aulas e as inscrições

GRADE DE AULAS

nº	aula	dia	horário	sala	vagas	instrutor
1	Spinning	seg	07:00	Sala 1	20	Paula
2	Spinning	qua	19:00	Sala 1	20	Paula
3	Funcional	ter	18:00	Área externa	25	Igor
4	Pilates	qui	08:00	Sala 2	12	Renata
5	Alongamento	sab	09:00	Sala 2	15	Renata



“Vagas” é a capacidade da aula. O instrutor vem pelo nome na ficha — no banco, entra o **código** dele.

FICHAS DE INSCRIÇÃO

nº	aluno	aula nº	data
1	Ana Beatriz	1	12/01/2026
2	Ana Beatriz	4	12/01/2026
3	Bruno	3	05/02/2026
4	Bruno	2	05/02/2026
5	Camila	2	16/03/2026
6	Camila	3	16/03/2026
7	Diego	1	03/05/2026
8	Elisa	4	20/06/2026
9	Elisa	2	20/06/2026
10	Fábio	3	02/08/2026
11	Heitor	1	02/10/2026
12	Heitor	4	02/10/2026

CONSOLIDAÇÃO • ETAPA 3 DE 4 • ALTER E REGRAS • 20 MIN

A dona voltou com novidades

O banco já está em uso, com dados. Nada de apagar e recriar: cada pedido é um `ALTER TABLE` no fim do `academia.sql`.

1. “Quero guardar o **e-mail** dos alunos — e cada e-mail é de um aluno só.”

2. “Abrimos a sala nova: a capacidade máxima passa de **30 para 35**.” *Dica: apague a regra antiga e crie outra. Qual é o nome dela?*

3. “Toda aula precisa ter **sala** definida.”

4. “Nos instrutores, chame **especialidade** de **modalidade**.”

AGORA TENTE QUEBRAR — CADA UM PRECISA SER RECUSADO

5. Excluir o plano Mensal • 6. Inscrever o Bruno de novo no Funcional • 7. Capacidade 50 no Funcional • 8. Situação “pausado” para a Gabriela. Anote, para cada um, a **regra** que barrou.

CONSOLIDAÇÃO • ETAPA 4 DE 4 • CONSULTAS COM JOIN • 40 MIN

As perguntas da dona

Um SELECT para cada pergunta, no `academia.sql`, com o comentário `-- Etapa 4 · 1`.

1. Os alunos com o nome do plano, em ordem alfabética.
2. A grade: aula, dia, horário, sala e instrutor, por horário.
3. Os inscritos no Spinning de segunda.
4. As aulas da Camila Freitas: aula, dia, horário e instrutor.
5. Os alunos do plano Anual que fazem Pilates.
6. Os alunos que não estão inscritos em nenhuma aula.
7. Os instrutores sem nenhuma aula.
8. As aulas sem nenhum inscrito.
9. O relatório completo: aluno, plano, aula, dia e instrutor, por aluno. *Cinco tabelas.*
10. Os alunos **trancados** que ainda ocupam vaga — com o código da inscrição. Depois, libere a vaga com DELETE.

PENSE E RESPONDA Na pergunta 9, a Gabriela aparece? Que mudança faria ela aparecer — e com que valores nas colunas da aula?

CONSOLIDAÇÃO • ENTREGA

Entrega e autoconferência

O QUE ENTREGAR

- [ ] `academia_der.png` e `academia_mer.png`, exportados do draw.io.
- [ ] `academia.sql`: DROP, CREATE, INSERT, ALTER, testes e consultas, cada parte com seu comentário.
- [ ] O banco `academia` no seu servidor — local ou Aiven.

O PROFESSOR VAI CONFERIR

- [ ] DER e MER coerentes com o briefing.
- [ ] Uma FK para cada ligação do MER.
- [ ] Cada regra do briefing como restrição — e testada.
- [ ] O script rodando do zero, sem erro.
- [ ] JOINs com ON certo e apelidos.

AUTOCONFERÊNCIA • DEU CERTO SE…

- Pergunta 1: **8** linhas • pergunta 9: **12**.
- Perguntas 3 e 5: **3** alunos cada.
- Perguntas 6, 7 e 8: **1** linha cada.
- Depois da faxina da 10: **11** inscrições.
- Os testes 5 a 8 da Etapa 3 deram **erro**.

Travou? Etapa 1 → Capítulos 1 e 2 • Etapa 2 → Seções 01 a 03 • Etapa 3 → ALTER e restrições • Etapa 4 → Seção 04 e a cola de sintaxe.

BANCO DE DADOS • CAPÍTULO CONCLUÍDO

Seu banco tem regras!

Entregue a consolidação da Academia Pulso. Dúvidas? Discord aberto.

DISCORD
discord.gg/wt5CVZZWJs

DRIVE
drive.google.com/drive/folders/1hOl0DaPeAor7gnhKBUIlZ5n8lLRDNvUY

GITHUB
github.com/TarikPonciano/Programador-de-Sistema-SENAC

postgresql.org/docs — CREATE, ALTER, JOIN

pgadmin.org/docs — Query Tool

w3schools.com/sql — exercícios de JOIN